

Module 07: Communication — Networks, APIs, and Multi-Agent Systems

Learning Objectives

1. Explain how **network protocols** (TCP/IP, HTTP) enable communication between digital agents.
2. Define an **API** (Application Programming Interface) as a structured boundary for agent-to-agent communication.
3. Analyze a multi-agent system (e.g., a multiplayer game server) as agents exchanging prediction errors.

Introduction

Digital agents rarely work alone. They communicate: your phone talks to a cell tower, which talks to a server, which talks to a database. This module explores how digital communication works — from the physical cables to the logical protocols — and frames it as multi-agent Active Inference: agents exchanging information to align their generative models.

Key Concepts

1. Network Protocols as Communication Rules

A **protocol** is a set of rules that two agents agree to follow when communicating. Without protocols, messages would be chaos.

- **TCP/IP**: Breaks data into packets, routes them across the Internet, and reassembles them. It handles prediction errors (lost packets) by requesting retransmission.
- **HTTP**: The language of the web. A browser sends a *request* ("GET /page.html") and the server sends a *response*. This request-response pattern is a prediction-error-correction cycle: the browser predicts what the page should look like and the server provides the actual content.

2. APIs: Markov Blankets for Software

An **API (Application Programming Interface)** defines exactly what information can pass between two software systems — and what is hidden. This is a Markov blanket for software.

Example: The Google Maps API lets your app request directions without knowing anything about Google's internal routing algorithms. Your app sends a request (origin, destination), and the API returns a route. The API boundary hides Google's vast internal complexity.

3. Multi-Agent Communication

In a multiplayer online game, thousands of players (agents) interact simultaneously. The game server maintains a shared world model. Each player's client sends actions (move, shoot, chat) and receives updates (other players' positions). Lag is a communication prediction error: your local model predicts the world has not changed, but the server update reveals it has.

Applications

- **Social Media Platforms:** Each user is an agent posting content. The platform's recommendation algorithm is a meta-agent determining which content reaches which user. The communication between user and algorithm shapes the information environment.
- **IoT (Internet of Things):** A smart home's thermostat, lights, and door lock communicate through a shared protocol (Zigbee, Matter). Each device has a local generative model ("it's 6 PM, usually someone is home"), and they coordinate to minimize collective free energy (comfortable temperature, appropriate lighting, locked doors).

Discussion Questions

1. When a web page fails to load (404 error), what "prediction error" did the browser experience?
2. How is an API similar to the Markov blanket of a biological organism? What information passes through, and what stays hidden?

Summary

Digital communication is governed by protocols that structure the exchange of information between agents. APIs create Markov blankets for software. Multi-agent systems align their generative models through continuous communication. Network prediction errors (lag, lost packets, 404s) drive the error-correction mechanisms that keep the digital world running.

References

- Kurose, J. F. & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach*. Chapters 1-2.